

Agenda

- Order By (- - -)
 - limit
 - Update
 - Delete
 - Joins
- } UD in CRUD

This Week:-

Tues → Joins 1

Thurs → Joins 2

Friday → Aggregate

Saturday → Subqueries

Next Week:-

Tues → Indexing

Thurs → Transactions

Friday → SD 1

Saturday → SD 2

Order By

```
select *title. from film  
where id >= 55  
order by release-year.
```

Execution order

where → ① put condition and pick all the cols.

order By → ② → sort them according to condition

select → ③ pick the cols required.

Order By DISTINCT Keyword

select distinct title from film order By release-year; - ①

→ Gives you an error ✗

Because this query will lead to ambiguous results.

(there will be multiple release-year for same title)

_____	2006
_____	2006
_____	2006
_____	2007
_____	2008
_____	2012

select distinct title from film order By title; - ②

→ Yes. ✓

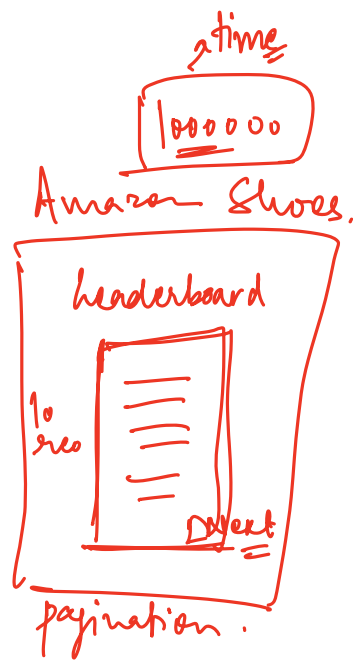
LIMIT

Select * from film limit 10;

Select * from students order By psp desc
limit 10;

Select * from students order By psp desc
limit 100 offset 10; $\Rightarrow$ 11 to 110
 $\rightarrow$ this query returns 10 rows starting from 11th.
 $100 + 10 \rightarrow \text{end}$

Select * from students order By psp desc
offset 10; $\rightarrow$ 11 to the end
 $\downarrow$ No end point



- ① WHERE
- ② ORDER BY
- ③ SELECT
- ④ limit (At the end)

Pseudocode:-

answer = [];

for each row in film:

if row.matches (where conditions):
 answer.append (row);
}

WHERE

length, title

answer.sort (column-names in order by clause)

ORDER BY

filtered-ans = []

for each row in answer:
filtered-ans.append(ans['rating'], row['release'])

SELECT

return filtered-answer[start-of-limit: end-of-limit]

UPDATE

UPDATE table-name SET column-name = value WHERE conditions;

e.g.

UPDATE film SET title = "Dark Does Not Rise" WHERE id=2;

Pseudo Code:-

```
for each row in film:
    if (row matches (conditions in where clause))
        row['title'] = "Dark Does Not Rise";
        row['release-year'] = 2006;
    }
```

UPDATE students SET phone-number = 819591 . where . . .

→ Dangerous.

Handle with Care.

DELETE

DELETE from table_name WHERE conditions;

delete from film where id=1;

Very cautious.

DELETE from film; ———

→ Deletes all rows.

Pseudocode:

```
for each row in film:
    if row matches (conditions in WHERE)
    {
        delete row;
    }
```

Delete v/s Truncate v/s Drop.

Truncate

Truncate film,

↳ This deletes all rows in a table. → 100% correct.

Much faster than delete

Why? This works internally by removing the entire table then recreates it without any rows.

Disadvantages of Truncate:-

1) It cannot be rolled back. } (Not possible to revert)
 ↳ undo

DROP

DROP table (film);

↳ deletes the film table.

Delete → removes rows in table

DROP → removes entire table.

Delete

→ Removes rows one by one from table

→ Delete is slower than truncate

→ Doesn't Reset the key

→ It can be rolled back

Truncate

→ Removes complete table then recreates without rows.

→ Faster than Delete

→ Resets the key.

(Starts with 1)

→ Cannot be rolled back

Drop

→ Removes complete table and table structure

→ Cannot be rolled back.

PK

id	name	batch
1	—	—
2	—	—
3	—	—
4	—	—
5	—	—
6	—	—

→ PK is not reset

Joins

Batches

batch-id | batch-name

Students

stud-id | first-name | last-name | batch-id

Student-name | batch-name

Batches

batch-id | batch-name

1 Batch A
2 Batch B
3 Batch C

Students

stud-id | first-name | last-name | batch-id

1 Bikash Kumar 1
2 Mukul Kumar 2
3 Aartee 2
4 Bhakti Thakkar 3
5 Durgesh Khos 1

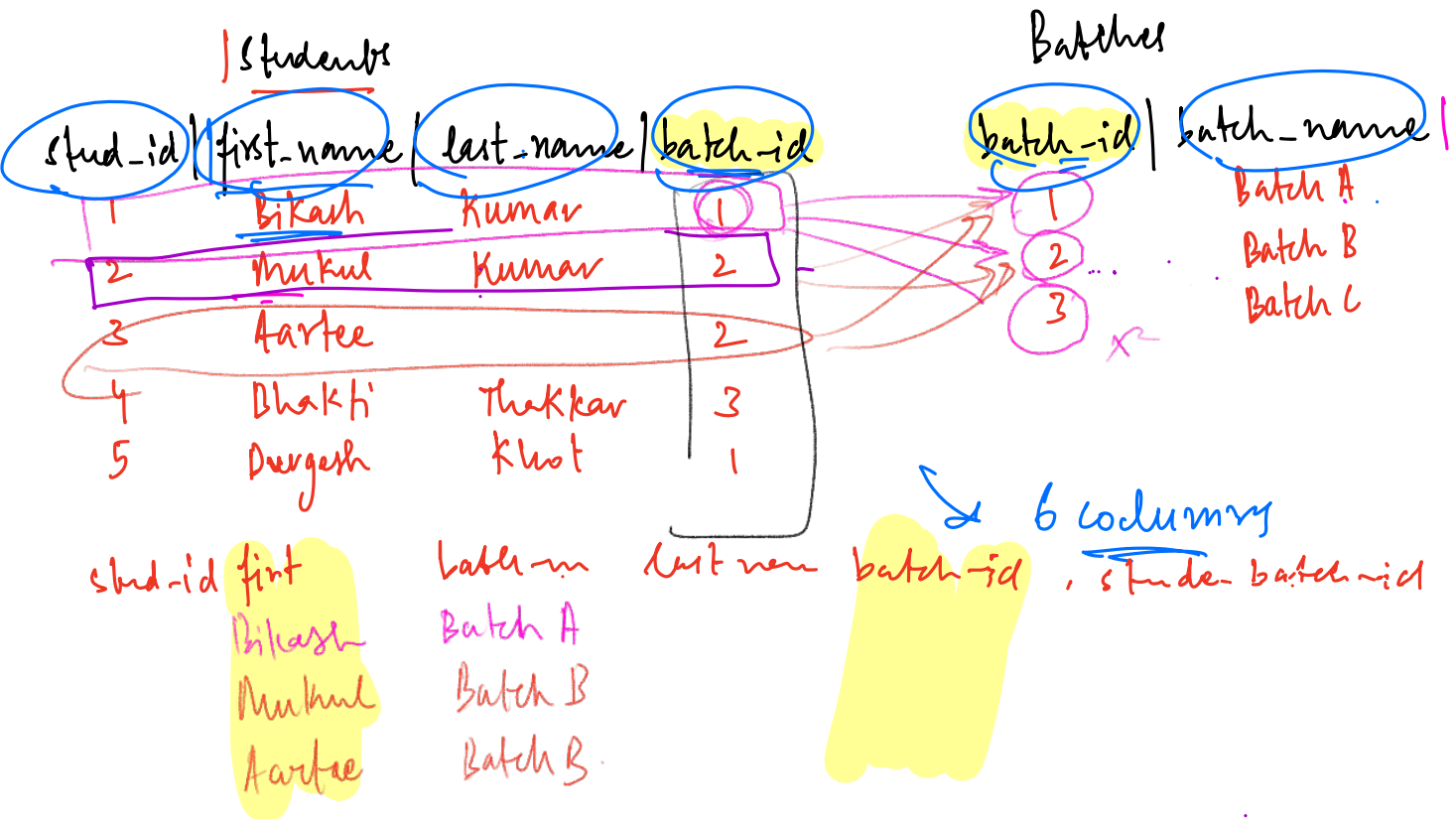
Student-name | batch-name

Bikash Batch A
Mukul Batch B
Aartee Batch B
Bhakti Batch C
Durgesh Batch A

Join condition

→ stitch based on batch-id.

```
for i=1
  for j=1
    for k=1
      for l=1
```



SQL Query

```

Select [ col. names ]
FROM table-name 1
JOIN table-name 2
ON condition.
  
```

```

select students.first-name, batches.batch-name
from students
join batches
on students.batch-id = batches.batch-id
  
```